

الدليل الشامل لأمان وتطوير تطبيقات Django

التحقق من الصحة، المصادقة، التفويض، وإدارة الجلسات

الطالب طارق العمري

مقدمة

لا يمكن النظر إلى مفاهيم أمان تطبيقات الويب على أنها وحدات منفصلة، بل هي نظام متكامل ومترابط يهدف إلى بناء تطبيقات قوية وآمنة وقابلة للصيانة.

يرتكز هذا النظام على ثلاثية أساسية للأمان: التحقق من الصحة (Validation)، والمصادقة (Authentication)، والتفويض (Authorisation)، مع آليات إدارة الحالة مثل الجلسات (Sessions) والرموز المميزة (Tokens).

← جامعة إب - علوم الحاسوب

فهرس المواضيع

يستعرض هذا العرض التقديمي الركائز الأساسية لأمان وتطوير تطبيقات Django مع التركيز على المفاهيم العملية والتطبيقية في بناء تطبيقات ويب آمنة وقابلة للصيانة.

إدارة الجلسات والحالة 🍪

المصادقة والتفويض 👤

التحقق من صحة البيانات 🛡️

شكراً 🙏

الاستنتاجات والتوصيات 📝

حزم الطرف الثالث 🧩

الجزء الأول: التحقق من صحة البيانات

أساس الأمان في أي تطبيق ويب متكامل هو آليات التحقق من صحة البيانات لضمان حماية التطبيق وقواعد البيانات من البيانات غير الصحيحة أو الضارة.

يتمثل التحقق من صحة البيانات في كونه مكونًا حيويًا للتأكد من المعلومات التي يقدمها المستخدم لضمان توافقها مع معايير محددة قبل معالجتها. يقدم Django نهجًا متعدد الطبقات للتحقق من الصحة، مما يوفر دفاعًا متعمقًا ضد البيانات غير الصالحة.

↳ أساس التطبيق الآمن

فلسفة التحقق في Django

يقدم Django نهجًا متعدد الطبقات للتحقق من صحة البيانات، مما يوفر دفاعًا متعمقًا ضد البيانات غير الصحيحة أو الضارة.

أهمية التحقق من جانب الخادم

على الرغم من أن التحقق من الصحة من جانب العميل باستخدام JavaScript يوفر تجربة مستخدم أفضل من خلال تقديم ملاحظات فورية، إلا أن التحقق من الصحة من جانب الخادم يظل هو المصدر النهائي للحقيقة. يجب ضمان سلامة البيانات على الخادم، حيث لا يمكن الوثوق بأي بيانات قادمة من العميل.

طبقات الدفاع المتعمقة

- **طبقة النموذج (Model Layer):** تمثل خط الدفاع الأخير، حيث يتم فرض قواعد العمل وسلامة البيانات على مستوى قاعدة البيانات.
- **طبقة النموذج (Form Layer):** تُستخدم للتعامل مع البيانات القادمة من نماذج HTML، وتوفر آلية غنية للتحقق من صحة المدخلات وتقديم رسائل خطأ واضحة للمستخدمين.
- **طبقة المُسلسلات (API Serializer Layer):** في إطار عمل Django REST Framework، تعمل المُسلسلات بشكل مشابه للنماذج ولكنها مصممة للتعامل مع بيانات واجهات برمجة التطبيقات.

دورة حياة التحقق في نماذج Django

تفصيل دقيق لمراحل التحقق من صحة البيانات في إطار Django، والتي تضمن تكامل البيانات وسلامتها قبل معالجتها وتخزينها في قاعدة البيانات. تعرف كيف تتدفق البيانات خلال مراحل التحقق المختلفة للوصول إلى بيانات نظيفة جاهزة للاستخدام.

١ to_python()

تحويل البيانات الأولية من أداة الإدخال إلى نوع بيانات Python صحيح. مثال: تحويل النصوص إلى أعداد عشرية في حالة FloatField.

٢ validate()

التحقق المخصص للحقل والذي لا يناسب وضعه في مُحقق قابل لإعادة الاستخدام. تتعامل مع البيانات بعد تحويلها ولا تعدل القيمة.

٣ run_validators()

تشغيل جميع محققات الحقل المحددة في وسيطة validators. تجمع جميع الأخطاء في ValidationError واحد.

٤ clean() (على مستوى الحقل)

المنظم للعملية، يستدعي الطرق الثلاث السابقة بالترتيب الصحيح وينتج قاموس cleaned_data الجاهز للمعالجة.

أساليب التحقق المتقدمة

بعد اكتمال التحقق الأساسي على مستوى الحقل، يوفر Django طريقتين متقدمتين لتنفيذ منطق تحقق أكثر تعقيداً وشمولية

+ التحقق الخاص بالحقل

تتيح طريقة `clean_<fieldname>()` التحقق من صحة حقل معين مع إمكانية الوصول إلى سياق النموذج بأكمله. المثال الكلاسيكي هو التحقق من أن اسم المستخدم فريد، وهو ما يتطلب استعلاماً من قاعدة البيانات. تعمل هذه الطريقة بعد التنظيف الأولي للحقل، ويجب أن تُرجع القيمة النظيفة.

+ التحقق المتقاطع بين الحقول

تلعب طريقة `Form.clean()` دوراً حاسماً في إجراء التحقق الذي يتطلب مقارنة حقول متعددة. مثال: التأكد من أن حقل "تأكيد كلمة المرور" يطابق حقل "كلمة المرور". يتم التعامل مع الأخطاء باستخدام `non_field_errors()` أو عبر `add_error('fieldname', 'الرسالة')` لربط الخطأ بحقل محدد.

التحقق على مستوى النموذج والمسلسلات

آليات التحقق في Django تختلف بين نماذج قاعدة البيانات وواجهات برمجة التطبيقات (API)، لكن كلاهما يوفر طبقة حماية أساسية للبيانات

DRF Serializers - تأمين واجهات API

تعكس آلية التحقق في Django REST Framework (DRF) تلك الموجودة في النماذج التقليدية، لكنها مصممة لبيانات API. من الناحية التصميمية، يتم إجراء جميع عمليات التحقق على مستوى المُسلسِل (Serializer) بأكمله، مما يوفر فصلاً أوضح للمسؤوليات وسلوكاً أكثر قابلية للتنبؤ.

Model.clean() - خط الدفاع الأخير

يعتبر التحقق على مستوى النموذج (Model.clean) المكان المثالي لفرض منطق العمل وسلامة البيانات عالمياً. ملاحظة مهمة: استدعاء model.save() لا يؤدي تلقائياً إلى استدعاء full_clean() مما قد يسمح بتجاوز قواعد التحقق! يجب تطبيق نمط تجاوز طريقة save() لمعالجة ذلك.

جدول مقارنة طبقات التحقق في Django

مقارنة شاملة بين طبقات التحقق المختلفة في Django والتي توفر آليات الدفاع المتعمق ضد البيانات غير الصحيحة أو الضارة. هذه المقارنة تساعد المطور على اتخاذ القرار المناسب حول مكان وضع منطق التحقق الخاص به.

الطبقة	الطريقة الأساسية	نقطة التنفيذ	حالة الاستخدام	النطاق
النموذج (Model)	<code>clean</code>	عند استدعاء <code>full_clean</code>	سلامة البيانات العالمية وقواعد العمل	كائن كامل
النموذج (Form)	<code>clean_<fieldname></code> , <code>clean</code>	عند استدعاء <code>is_valid</code>	معالجة أخطاء واجهة المستخدم	حقل واحد، حقول متعددة
مُستطيل DRF	<code>validate_<fieldname></code> , <code>validate</code>	عند استدعاء <code>is_valid</code>	فك تسلسل بيانات API والتحقق منها	حقل واحد، حقول متعددة

النموذج خط الدفاع الأخير

يمثل التحقق على مستوى النموذج المكان المناسب لفرض قواعد العمل وسلامة البيانات بغض النظر عن طريقة الإنشاء. لا يتم استدعاء `model.full_clean` تلقائياً عند الحفظ.

النماذج معالجة المدخلات

توفر آلية غنية للتحقق من صحة المدخلات من نماذج HTML وتقديم رسائل خطأ واضحة للمستخدمين. نقطة القوة: التعامل مع تفاعل المستخدم وتقديم تجربة مستخدم أفضل.

DRF تأمين واجهات API

تعكس المُستطيلات في DRF آلية التحقق الموجودة في نماذج Django، لكنها مصممة خصيصاً لعمليات التسلسل وفك التسلسل للبيانات مثل JSON وتأمين واجهات برمجة التطبيقات.

الجزء الثاني: المصادقة والتفويض

حماية هوية المستخدم وضبط صلاحياته للوصول من ركائز أمان الأنظمة الحديثة. في Django، توجد منظومة متكاملة لإدارة المصادقة والتفويض على مستوى المستخدمين والمجموعات والعروض.

المصادقة (Authentication) تجيب على سؤال "من أنت؟" بينما التفويض (Authorization) يجيب على "ماذا يُسمح لك أن تفعل؟" - نظام Django المتكامل (django.contrib.auth) يدير كلا الجانبين بكفاءة وأمان.

التمييز بين المصادقة والتفويض

المصادقة والتفويض مفهومان مترابطان لكنهما مختلفان في أمان التطبيقات، ويوفر Django نظامًا شاملاً للتعامل مع كليهما

نظام المصادقة في Django

يوفر `django.contrib.auth` نظامًا متكاملًا للمصادقة والتفويض يتكون من:

- نموذج `User`: نموذج متكامل للمستخدمين
- تجزئة كلمات المرور: نظام آمن وقابل للتكوين
- خلفيات (`backends`): واجهات للمصادقة قابلة للتوصيل والتخصيص
- نظام أذونات: يدعم الأذونات الدقيقة والمجموعات

يُنفذ هذا النظام ثلاث وظائف أساسية: `login`، `authenticate()` و `logout()`

المفاهيم الأساسية



التفويض (Authorization)

عملية تحديد صلاحيات المستخدم، تجيب على سؤال: "هل يُسمح لك بفعل هذا؟"
تتحقق من أذونات المستخدم بعد التأكد من هويته.



المصادقة (Authentication)

عملية التحقق من هوية المستخدم، تجيب على سؤال: "من أنت؟"
تؤكد أن المستخدم هو من يدعي أنه هو باستخدام كلمة مرور أو طرق أخرى.

نظام المصادقة المدمج في Django

يقدم Django نظامًا شاملاً ومتكاملاً للمصادقة من خلال حزمة `django.contrib.auth` التي تتعامل مع هويات المستخدمين وإدارة الجلسات وتأمين كلمات المرور. يوفر النظام واجهات برمجية سهلة الاستخدام للتحقق من هوية المستخدمين وإدارة حساباتهم وتسجيل الدخول والخروج.

👤 نموذج User

نموذج كامل لتمثيل المستخدمين يتضمن حقول أساسية مثل اسم المستخدم وكلمة المرور والبريد الإلكتروني والاسم الأول والأخير، بالإضافة إلى طرق مساعدة مثل `create_user` و `create_superuser`.

🔑 تجزئة كلمات المرور

نظام متقدم وقابل للتكوين لتشفير وتجزئة كلمات المرور باستخدام PBKDF2 بشكل افتراضي. يجب استخدام `set_password()` لتعيين كلمات المرور بدلاً من التعيين المباشر.

➡ وظائف المصادقة

ثلاث وظائف أساسية: `authenticate()` للتحقق من البيانات، `login()` لإنشاء جلسة للمستخدم، و `logout()` لإنهاء الجلسة. تعمل هذه الوظائف معًا لتوفير دورة حياة كاملة للمصادقة.

🔗 خلفيات المصادقة

نظام قابل للتوصيل يسمح بإضافة طرق مصادقة مخصصة أو خارجية مثل OAuth أو LDAP. يمكن تكوين خلفيات متعددة في `AUTHENTICATION_BACKENDS` في إعدادات المشروع.

تأمين العروض والقوالب

يوفر Django أدوات متعددة لتأمين العروض والقوالب بناءً على حالة المصادقة والأذونات، سواء كانت العروض قائمة على الدوال أو الفئات

🔑 العروض القائمة على الدوال

مُخزّنات تحمي الوصول للدوال:

- `@login_required` - يضمن أن المستخدم مسجل دخوله للوصول إلى العرض. إذا لم يكن كذلك، يتم إعادة توجيهه إلى صفحة تسجيل الدخول.
- `@permission_required` - يتحقق مما إذا كان لدى المستخدم إذن معين مثل: `'app.change_model'`
- `@user_passes_test` - يسمح بتحديد دالة اختبار مخصصة للتحكم في الوصول

🔑 العروض القائمة على الفئات والقوالب

للعروض القائمة على الفئات:

- `LoginRequiredMixin` - وظيفة مماثلة لـ `@login_required`
- `PermissionRequiredMixin` - وظيفة مماثلة لـ `@permission_required`

في القوالب:

التحقق من الأذونات مباشرة في القوالب باستخدام:

```
{% if perms.app_label.change_model %}
```

هذا يسمح بإظهار عناصر واجهة المستخدم بشكل شرطي حسب الصلاحيات

المصادقة والأذونات في Django REST Framework

يقدم Django REST Framework (DRF) نظامًا قويًا للمصادقة والأذونات مصممًا خصيصًا لواجهات برمجة التطبيقات (APIs) ويفصل المسؤوليات بشكل أكثر وضوحًا

الأذونات على مستوى الكائن

للتحكم بشكل أكثر دقة، يمكن إنشاء فئة أذونات مخصصة بوراثة BasePermission وتنفيذ طريقتين أساسيتين:

- **has_permission:** للتحقق على مستوى العرض (القائمة/الكل)
- **has_object_permission:** للتحقق على مستوى الكائن الفردي

تحسين الأداء: في عروض القائمة، يستدعي DRF has_permission فقط، وفي العروض التفصيلية، يستدعي كلا الطريقتين، مما يمنع استعلامات قاعدة البيانات غير الضرورية.

فئات الأذونات المدمجة

يوفر Django REST Framework عدة فئات أذونات مدمجة للتحكم في الوصول إلى واجهات API:

- **AllowAny:** يسمح بالوصول غير المقيد للجميع
- **IsAuthenticated:** يسمح بالوصول للمستخدمين المسجلين فقط
- **IsAdminUser:** يقصر الوصول على المستخدمين الإداريين
- **IsAuthenticatedOrReadOnly:** قراءة فقط للزوار، وصول كامل للمستخدمين المسجلين

الجزء الثالث إدارة الحالة والجلسات

الخيار بين الجلسات ذات الحالة أو الرموز المميزة (مثل JWT) قرار معماري جوهري يؤثر على بنية الأمان وقابلية التوسع وأداء تطبيقات الويب الحديثة.

تتباين استراتيجيات إدارة الحالة بين النهج التقليدي ذو الحالة (Stateful) الذي تمثله الجلسات في Django، والنهج الحديث عديم الحالة (Stateless) الذي تمثله الرموز المميزة (Tokens) ورموز JWT. يؤثر اختيار أي منهما على الأمان وقابلية التوسع وتجربة المستخدم.

٤ علوم الحاسوب - مستوى ٤

جلسات Django التقليدية

آلية ذات حالة (Stateful) حيث يقوم الخادم بإنشاء وتخزين سجل الجلسة، ويرسل للعميل ملف تعريف ارتباط (Cookie) يحتوي على معرف الجلسة. تعتمد هذه الطريقة على تخزين البيانات في الخادم وإدارتها مركزياً.



قاعدة البيانات (db)

الخيار الافتراضي لـ Django. تخزن الجلسات في جدول خاص بقاعدة البيانات. يوفر استمرارية البيانات ويناسب معظم التطبيقات.

الذاكرة المخزنة (cache)

سريعة جداً ولكنها غير دائمة. يمكن فقد بيانات الجلسات عند إعادة تشغيل الخادم أو عند امتلاء الذاكرة.

قاعدة البيانات المخزنة (cached_db)

تجمع بين سرعة الذاكرة المخزنة واستمرارية قاعدة البيانات. القراءات من الذاكرة والكتابة في قاعدة البيانات.

الملفات (file)

تخزن الجلسات في ملفات على نظام الملفات. بسيط في التطبيق ولكنه قد يكون أبطأ وأقل قابلية للتوسع.

TokenAuthentication في DRF

حل مباشر وبسيط لكنه "شبه عديم الحالة" حيث يتطلب بحثاً في قاعدة البيانات للتحقق من صحة الرمز. يتم إرسال الرمز في ترويسة
 Authorization: Token <token>

تركيب رموز JWT

الترويسة . الحمولة . التوقيع

الترويسة تحدد نوع الرمز وخوارزمية التوقيع، والحمولة تحتوي على المطالبات مثل معرف المستخدم ووقت انتهاء الصلاحية، والتوقيع يضمن سلامة البيانات.

نموذج Refresh Token و Access Token

رمز الوصول: قصير العمر (5-15 دقيقة)، يستخدم لطلبات API. **رمز التحديث:** طويل العمر (أيام/أسابيع)، يستخدم فقط لتجديد رمز الوصول عند انتهاء صلاحيته. يقلل هذا النموذج من المخاطر الأمنية مع الحفاظ على تجربة مستخدم سلسة.

رموز التوكن و JWT للمصادقة عديمة الحالة

تتيح المصادقة عديمة الحالة (Stateless) بناء أنظمة أكثر قابلية للتوسع وخاصة في بيئات الخدمات المصغرة وتطبيقات الويب أحادية الصفحة (SPA). يقدم Django REST Framework نظام TokenAuthentication البسيط، بينما تعتبر رموز JWT الحل الأكثر قوة وشمولاً للمصادقة في واجهات برمجة التطبيقات الحديثة.

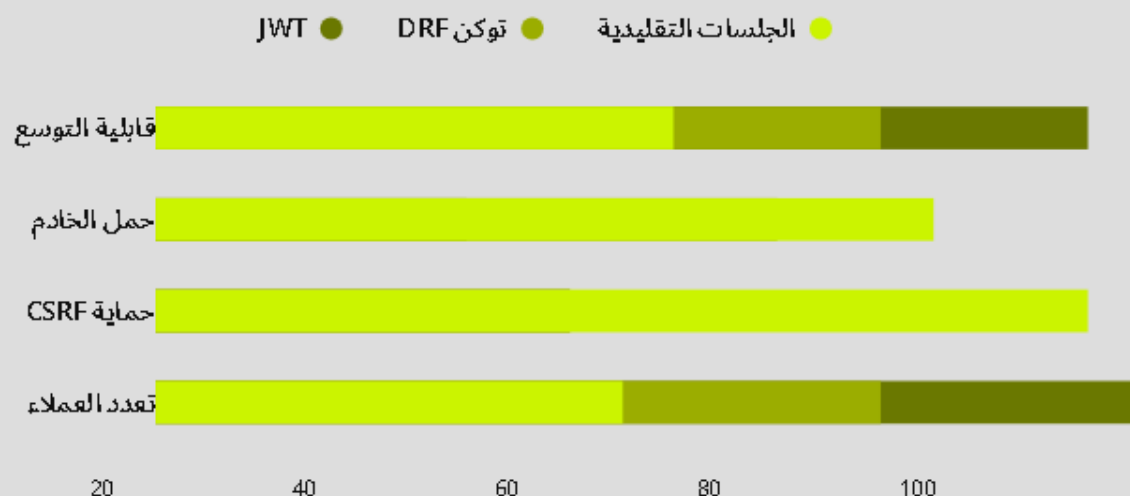
جدول مقارنة استراتيجيات المصادقة

مقارنة بين الأساليب المختلفة للمصادقة في تطبيقات Django، مع تحليل لنقاط القوة والضعف والاستخدامات المثلى لكل أسلوب.

الجلسات تناسب تطبيقات الويب التقليدية بينما JWT يعتبر الخيار الأفضل لتطبيقات SPA والجوال والخدمات المصغرة.

JWT يتميز بقابلية توسع ممتازة وحمل أقل على الخادم، لكن الجلسات توفر حماية CSRF مدمجة.

يجب اختيار استراتيجية المصادقة بناءً على متطلبات المشروع ونوع العملاء المستهدفين.



حزم الطرف الثالث الفعالة

django-extensions +

مجموعة شاملة من الأدوات المساعدة وأوامر الإدارة التي تعزز إنتاجية المطور. تشمل الميزات الرئيسية shell_plus للاستيراد التلقائي للنماذج، وrunserver_plus مع مصحح أخطاء Werkzeug، وgraph_models لإنشاء مخططات مرئية.

django-widget-tweaks +

تسمح هذه الحزمة بتعديل سمات حقول النماذج مباشرة في القالب، دون الحاجة إلى كتابة كود Python. يمكنك بسهولة إضافة الأصناف (classes) والسمات الأخرى، مما يعزز تجربة المستخدم وتصميم النماذج في تطبيقك.



الخلاصة والتوصيات

التحقق من الصحة من جانب الخادم ليس خياراً - بل ضرورة حتمية لضمان سلامة وموثوقية البيانات ✓

استخدام الجلسات للتطبيقات التقليدية واستخدام JWT لتطبيقات SPA وواجهات API والخدمات المصغرة ✓

الاستفادة من النظام البيئي في Django (مثل django-extensions و django-widget-tweaks) لتعزيز الإنتاجية ✓

تقديم الطالب: طارق العمري